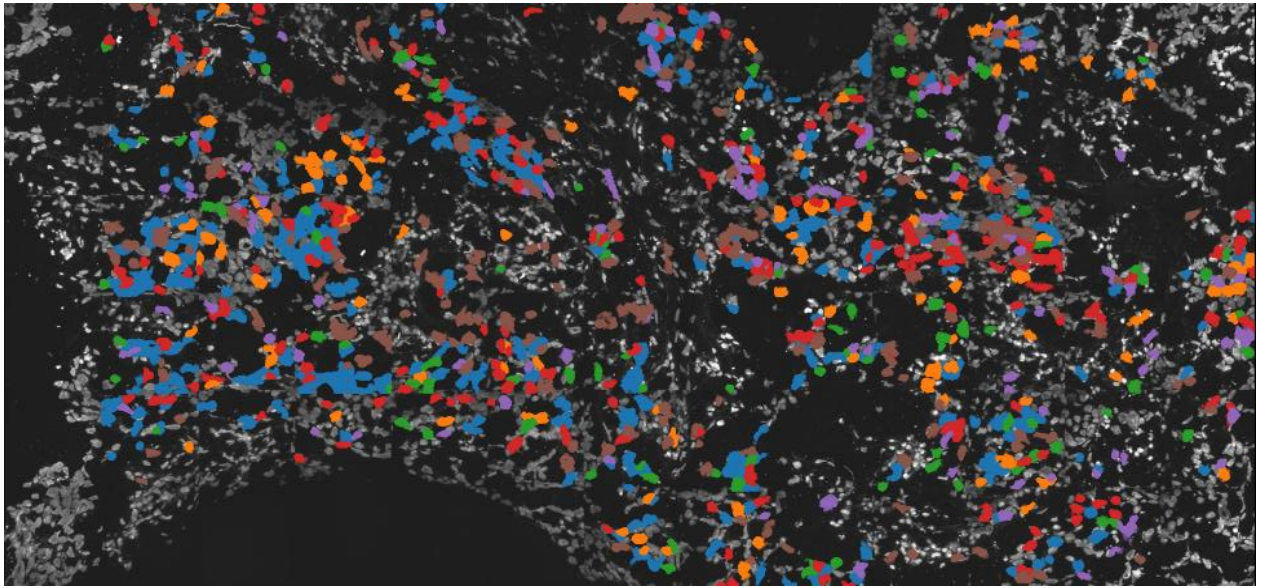
Neuro Genomics

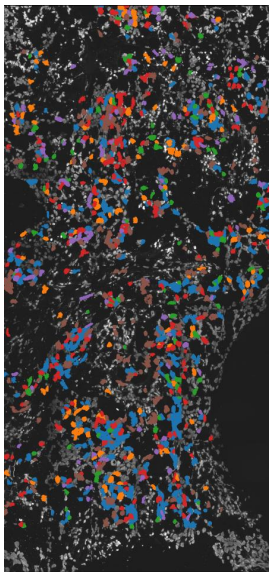
Michal Danino Levi
michal.danino@live.biu.ac.il
Room 342



Gene 1	Gene 2	Gene 3	Gene 4	Gene 5	Gene 6	Gene 7	Gene 8	Gene 9	Gene 10
9	56	84	13	4	207	92	105	24	79

Three main steps in the creation of the expression vector:

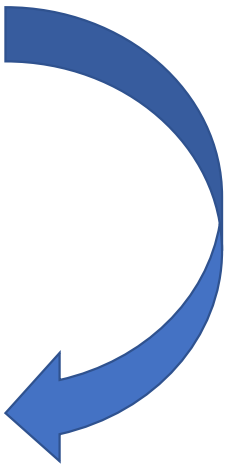
1) Isolation of RNA molecules from the tissue
2) Amplification and sequencing of RNA molecules
3) Alignment of RNA sequences against the genes/genome



Gene 1	Gene 2	Gene 3	Gene 4	Gene 5	Gene 6	Gene 7	Gene 8	Gene 9	Gene 10
9	56	84	13	4	207	92	105	24	79

Three main steps in the creation of the expression vector:

1) Isolation of RNA molecules from the tissue
2) Amplification and sequencing of RNA molecules
3) Alignment of RNA sequences against the genes/genome



- **FASTQ format**
- **Regular Expressions in Python**

FASTQ format

The quality score of the base is called the Phred score, and gives a measure of how accurate was the sequencing in that base.

Q = -log10(P-value)

For Illumina 1<=Q<=40,
Q<15 is considered very bad quality and Q>30 is considered ok.

FASTQ format

Each Q value is represented by one ASCII letter.

```
LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL..........................................
PPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
0.2.....................26...31........41
0.................20........30........40........50.....................................93
```

The format is:

@ read title
SEQUENCE
+ read title
QUALITY

```
@SRR001666.1 071112_SLXA-EAS1_s_7:5:1:817:345 length=36
GGGTGATGGCCGCTGCCGATGGCGTCAAATCCCACC
+SRR001666.1 071112_SLXA-EAS1_s_7:5:1:817:345 length=36
IIIIIIIIIIIIIIIIIIIIIIIIIIIII9IG9IC
```

Regular Expressions in Python

You can ask questions such as “Does this string match the pattern?”, or “Is there a match for the pattern anywhere in this string?”.

Example:

- Print a list of all matches of “ai”.
- Search the string to see if it starts with "The" and ends with "Spain“.

```
import re
txt = "The rain in Spain"
x = re.findall("ai", txt)
['ai', 'ai']
x = re.search("^The.*Spain$", txt)
<re.Match object; span=(0, 17), match='The rain in
Spain'>
```

https://www.w3schools.com/python/python_regex.asp , https://docs.python.org/3/howto/regex.html#regex-howto , https://docs.python.org/lmth.er/yrarbil/3

Regular Expressions in Python

Function	Description
match ( )	Determine if the RE matches at the beginning of the string.
search ( )	Return a Match object if there is a match anywhere in the string.
findall ( )	Return a list containing all matches.
.group( )	Return the string matched by the RE.
.start( )	Return the starting position of the match.
.end( )	Return the ending position of the match.
.span( )	Return a tuple containing the (start, end) positions of the match.

Regular Expressions in Python

Character	Description	Example3
[ ]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
$	Ends with	"planet$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{ }	Exactly the specified number of occurrences	"he.{2}o"
|	Either or	"falls|stays"
( )	Capture and group	

Regular Expressions in Python

Character	Description	Example
\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"
\n	End of line	"\n"
\t	Tab	"\t"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white space character	"\S"
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Regular Expressions in Python

Set	Description
[arn]	Returns a match where one of the specified characters (a, r, or n) is present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case
[+]	In sets, +, *, ., |, (), $,{} has no special meaning, so [+] means: return a match for any + character in the string

Regular Expressions in Python

Lookahead and lookbackwards	Description	Example
(?=…)	Matches if ... matches next but doesn’t consume any of the string. This is called a lookahead assertion.	Isaac (?=Asimov) will match 'Isaac ' only if it’s followed by 'Asimov'.
(?!...)	Matches if ... doesn’t match next.	Isaac (?!Asimov) will match 'Isaac ' only if it’s not followed by 'Asimov'.
(?<=…)	Matches if the current position in the string is preceded by a match for ... That ends at the current position. This is called a positive lookbehind assertion.	
(?<!...)	Matches if the current position in the string is not preceded by a match for .... This is called a negative lookbehind assertion.	

Regular Expressions in Python

String: “at” , “hat” , “cat” , “bat”

Examples:

- .at –
- matches any three-character string ending with "at", including "hat", "cat", "bat" and "at".
- [hc]at –
- matches "hat" and "cat".
- [^b]at –
- matches all strings matched by .at except "bat".
- ^[hc]at –
- matches "hat" and "cat", but only at the beginning of the string or line.
- [hc]?at –
- matches "at", "hat", and "cat".
- \[.\] –
- matches any single character surrounded by "[" and "]" since the brackets are escaped.

Regular Expressions in Python

Exercise:

- \d is equivalent to _____
- \D is equivalent to _____
- What is the difference between “ca*t” and “ca+t” in “ct”?
- What is the difference between “ca*t” and “ca.*t” in “ct” / “cart”?
- txt = “Neuro Genomics course 2025”
  - Print the year of the course.
  - Print the name of the course (without the word “course”).
- txt = “spam-egg”
  Search the string to look for a word following a hyphen (-).
- txt = “abc\def”
  Print the positions (start and end) of “\”.

Regular Expressions in Python

- \d is equivalent to [0-9]
- \D is equivalent to [^0-9]
- <re.Match object; span=(0, 2), match='ct'> / None
- <re.Match object; span=(0, 2), match='ct'> / None
  None / <re.Match object; span=(0, 4), match='cart'>
- re.findall("[0-9]+",txt) / re.search("[0-9]+",txt)
  re.search(".*(?=course)",txt)
- re.search("(?<=-)\w+", "spam-egg")
- re.search("\\\\", "abc\def").span( )